2MAMBA2FURIOUS: LINEAR IN COMPLEXITY, COMPETITIVE IN ACCURACY

A PREPRINT

Gabriel Mongaras
Lyle School of Engineering
Southern Methodist University
Dallas, TX 75205
gabriel@mongaras.com

Eric C. Larson
Lyle School of Engineering
Southern Methodist University
Dallas, TX 75205
eclarson@smu.edu

May 2, 2026

ABSTRACT

Linear attention transformers have become a strong alternative to softmax attention due to their efficiency. However, linear attention tends to be less expressive and results in reduced accuracy compared to softmax attention. To bridge the accuracy gap between softmax attention and linear attention, we manipulate Mamba-2, a very strong linear attention variant. We first simplify Mamba-2 down to its most fundamental and important components, evaluating which specific choices make it most accurate. From this simplified Mamba-2 variant (Mamba-2S), we improve the A-mask and increase the order of the hidden state, resulting in a method, which we call 2Mamba, that is nearly as accurate as softmax attention, yet much more memory efficient for long context lengths. We also investigate elements to Mamba-2 that help surpass softmax attention accuracy. Code is provided for all our experiments [1] [2].

1 Introduction

Transformers have become the standard architectural backbone for modern language models. The core of the transformer is softmax attention, which routes information between all tokens in a set. While softmax attention is a highly accurate model component, it imposes quadratic complexity with respect to the sequence length during training. When causal, softmax attention has linear complexity during inference. Although the complexity of softmax attention cannot be reduced to linear complexity during training, algorithms such as Flash Attention (Dao et al. [2022]) make softmax attention more computationally efficient by utilizing highly efficient CUDA kernels (tiling). However, even with a highly optimized kernel, the underlying algorithm still retains quadratic complexity in FLOP count.

Linear attention algorithms (Katharopoulos et al. [2020]) aim to reduce the complexity of softmax attention by replacing the exponential nonlinearity in the softmax function with a decomposable kernel function. These algorithms are linear during training and constant during inference, making them much more desirable than softmax attention, solely based on algorithmic complexity. Another advantage of linear attention is that it can be implemented as an RNN (Rumelhart et al. [1986]), which increases efficiency for long sequence inference. While linear attention has linear complexity during training and constant complexity during inference, the accuracy is consistently worse than that of softmax attention.

More recent works in NLP applications make naive linear attention more expressive. Two such prominent works are Mamba (Gu and Dao [2024]), which adds semi-separable decay components to attention, and DeltaNet (Yang et al. [2024b]), which uses the delta rule (Schlag et al. [2021]), modeling the linear attention update rule as a form of gradient descent. The accuracy of these algorithms is significantly better than vanilla linear attention but still falls short of full softmax attention. Motivated by this finding, we aim to build on these efficient attention alternatives and increase

[1] https://github.com/gmongaras/2Mamba2Furious
[2] https://huggingface.co/collections/gmongaras/2mamba2furious-linear-in-complexity

model accuracy closer to that of softmax attention. Specifically, we (1) employ Mamba-2 (Dao and Gu [2024]) as a base model, (2) isolate the important components of the algorithm, and (3) use a higher order hidden state to reach softmax-level accuracy while keeping the complexity of the model linear. We build upon the work of (Mongaras and Larson [2025]), who showed that higher order hidden states with linear attention get closer to softmax-level accuracy. Via the Taylor expansion of the exponentiated query-key inner product, one obtains softmax attention from linear attention with higher order hidden states. We leverage this finding to improve the expressivity of Mamba-2. Additionally, we show that using an exponentiated query-key inner product improves the Mamba-2 model and outperforms softmax attention, albeit at the cost of requiring a KV cache. We also explore the similarity of this approach to the forgetting transformer (Lin et al. [2025]).

2 Background

2.1 Softmax Attention

(Bahdanau et al. [2016]) introduced softmax attention and (Vaswani et al. [2017]) popularized the use of softmax attention by creating the transformer architecture for natural language translation. Since then, the transformer can be found in most modern machine learning architectures and has been used in a variety of applications such as in computer vision (Dosovitskiy et al. [2021]), reinforcement learning (Chen et al. [2021]), robotics (Brohan et al. [2023]), generative image models (Ramesh et al. [2022]) (Kong et al. [2025]) (Radford et al. [2023]), and medicine (Jumper et al. [2021]), among many other applications. The softmax attention formulation is found in equation 1 (without the $\frac{1}{\sqrt{d_k}}$ term for simplicity).

$$Q = XW_Q \in \mathbb{R}^{H,N,d_h} \quad K = XW_K \in \mathbb{R}^{H,N,d_h} \quad V = XW_V \in \mathbb{R}^{H,N,d_h}$$
$$O = \text{softmax}\left(QK^T + M\right)V \quad = \quad \frac{\exp\left(QK^T + M\right)V}{\sum_j \exp(QK^T + M)} \tag{1}$$

The exponential term forces a quadratic FLOP count with respect to the sequence length during training and a linear FLOP count during inference, which is not ideal for long context applications. Several methods have been developed to reduce the quadratic bottleneck and make computing the quantity more efficient. Most notably, Flash Attention (Dao et al. [2022]) computes softmax attention in tiles for better GPU utilization. Although Flash Attention is more efficient than computing softmax attention in native PyTorch, the FLOP count remains quadratic.

2.2 Linear attention (naive computation)

Linear attention is derived by taking multi-head softmax attention and replacing the softmax nonlinearity function with a kernel function $\exp(QK^T) \Rightarrow K(Q,K) = \phi(Q)\phi(K)^T$, as shown in Algorithm 1. By making this substitution, linear attention can either be computed by first taking the inner product of the queries and keys (equation (2)) or by first taking the inner product of the keys and values (equation (3)). The latter has quadratic complexity in the dimension while the former, like softmax attention, has quadratic complexity in the sequence length. One note is that adding the causal mask requires a special kernel to make causal linear attention computationally efficient, otherwise the training complexity in naive torch is similar to softmax attention as seen in Algorithm 1. While linear attention is desirable

due to its linear complexity, naive linear attention falls short of softmax attention performance, as shown in Figure 1.

Algorithm 1 Linear attention

Require: $X \in \mathbb{R}^{N,d}$
Require: $W_{QKV} \in \mathbb{R}^{d,3\cdot(H\cdot d_h)}$
Require: $W_{out} \in \mathbb{R}^{(H\cdot d_h),d}$
Ensure: $\phi(\cdot) = \text{ReLU}(\cdot)$

$$
\begin{aligned}
QKV &= XW_{QKV} && \in \mathbb{R}^{N,3\cdot(H\cdot d_h)} \\
Q,K,V &= \text{split}(QKV) && \in \mathbb{R}^{H,N,d_h} \\
y &= ((\phi(Q)\phi(K)^T \odot M)V) && \in \mathbb{R}^{H,N,d_h} \\
N &= \sum_j (\phi(Q)\phi(K)^T \odot M) && \in \mathbb{R}^{H,N} \\
y_n &= y/N && \in \mathbb{R}^{H,N,d_h} \\
y'_n &= \text{vec}(y_n)_{(H,N,d_h)\longrightarrow(N,H\cdot d_h)} && \in \mathbb{R}^{N,H\cdot d_h} \\
o &= y'_n W_{out} && \in \mathbb{R}^{N,d}
\end{aligned}
$$

$$Q = XW_Q \quad K = XW_K \quad V = XW_V$$

$$O = \frac{\left[\phi(Q)\phi(K)^T\right]V}{\sum_j \phi(Q)\phi(K)^T} \tag{2}$$

$$= \frac{\phi(Q)\left[\phi(K)^T V\right]}{\phi(Q)\sum_j \phi(K)^T} \tag{3}$$

2.3 Mamba-2

Several approaches have attempted to improve the accuracy of linear attention while retaining linear complexity, such as performer (Choromanski et al. [2021]), cosformer (Qin et al. [2022]), hedgehog (Zhang et al. [2024]), RetNet (Sun et al. [2023]), gated linear (Yang et al. [2024a]), and DeltaNet (Yang et al. [2024b]), among others. One algorithm that is both efficient and improves linear attention significantly is Mamba-2 (Dao and Gu [2024]). In the description below, we examine Mamba-2 purely from an architectural perspective, not from the efficiency gained from the associative scan algorithm.

The original Mamba (Gu and Dao [2024]) architecture was derived from a state space model (SSM) (Kalman [1960]) (Gu et al. [2022]), which are not expressive on their own due to having time-independent queries, keys, and values (referred to as C, and B, and x in the SSM literature). Mamba improves SSM expressiveness by making the query, key, and value matrices time-dependent. Mamba-2 further improves efficiency and expressiveness by introducing the associative scan algorithm and decay mask (A mask). The resulting algorithm can be effectively formulated as linear attention with a decay mask (section 3 expands this assertion and isolates elements of Mamba-2 that are most expressive). From this perspective, Mamba-2 takes linear attention and makes it more expressive through clever architectural elements and parallelization via associativity.

The complete Mamba-2 algorithm is found in GitHub[3] and has some additions not described fully in the paper (Dao and Gu [2024]). That is, the Mamba-2 codebase builds in several inductive biases. The full Mamba model with all biases included from this codebase is written out in algorithms 2 and 3. Note that the head and sequence dimensions are arbitrarily swapped to simplify the presented algorithm, though the swaps can be inferred via the explicit shapes. To make the comparison with other algorithms easier, as noted in (Dao and Gu [2024]), we employ the Q, K, and V notation, rather than the equivalent C, B, and x notation (respectively). Additional simplifications were made such as using a single head group, however like with softmax attention, one can have head groups akin to MQA in attention (Ainslie et al. [2023]).

As seen in Algorithm 2 and Algorithm 3, Mamba-2 has numerous elements beyond just a decay mask. Many of the choices seem arbitrary or underexplored and add complexity without much investigation or motivation. We question if the added complexity is necessary for Mamba-2 to outperform vanilla linear attention. To analyze the components that make Mamba-2 superior (and propose changes), we break Mamba-2 into its components and build up a simplified algorithm, which we name Mamba-2S, from these singular components.

[3] https://github.com/state-spaces/mamba

Algorithm 2 Mamba-2 Setup

$$
\begin{aligned}
H &= \text{num heads}, \quad d_h = \text{head (inner) dim}, \quad N = \text{seq. length}, \\
\mathcal{U}[\cdot] &= \text{Uniform Dist.}, \qquad d_{ssm} = H * d_h \\
d_{conv} &= 3 \cdot d_{ssm}, \quad dt_{min} = 0.001, \quad dt_{max} = 0.1, \quad l_{min} = \log(dt_{min}), \quad l_{max} = \log(dt_{max}) \\
A_{log} &= \log(\mathcal{U}[1, 16]) \in \mathbb{R}^H, \quad D = \text{ones}(H * d_h) \in \mathbb{R}^{H \cdot d_h} \\
dt_{init} &= \exp(x) \in \mathbb{R}^H \quad \text{where} \quad x \sim \mathcal{U}[l_{min}, l_{max}] \\
dt_{bias} &= \text{softplus}^{-1}(dt_{init}) \quad = dt_{init} + \log(1 - \exp(-dt_{init})) \in \mathbb{R}^H [-4pt]
\end{aligned}
$$

Algorithm 3 Mamba-2 Forward Pass

Require: $h \in \mathbb{R}^{N,d}, \quad W_z \in \mathbb{R}^{d,H \cdot d_h}, \quad W_{QKV} \in \mathbb{R}^{d,d_{conv}} \quad W_{dt} \in \mathbb{R}^{d,H}$
$W_{out} \in \mathbb{R}^{H \cdot d_h, d}, \quad D \in \mathbb{R}^{H \cdot d_h}, \quad A_{log} \in \mathbb{R}^H, \quad dt_{bias} \in \mathbb{R}^H$

$$
\begin{aligned}
Q, K, V &= \sigma(\text{conv_1d}(h \cdot W_{QKV})) && \in \mathbb{R}^{H,N,d_h} \\
dt_P &= h \cdot W_{dt} && \in \mathbb{R}^{H,N} \\
Z &= h \cdot W_z && \in \mathbb{R}^{H,N,d_h} \\
A &= -\exp(A_{log}) && \in \mathbb{R}^H \\
dt &= \text{softplus}(dt_P + dt_{bias}) && \in \mathbb{R}^{H,N} \\
D_{res} &= V \odot D, \qquad V_{dt} = V \odot dt && \in \mathbb{R}^{H,N,d_h} \\
A^{CS} &= \text{cumsum}(A \odot dt) && \in \mathbb{R}^{H,N} \\
A^M &= \exp(A^{CS} - (A^{CS})^T) \quad \text{such that } A^M_{ij} = \exp(A^{CS}_i - A^{CS}_j) && \in \mathbb{R}^{H,N,N} \\
M_{ij} &= \begin{cases} 1, & \text{if } i \geq j \\ 0, & \text{if } i < j \end{cases} && M \in \mathbb{R}^{H,N,N} \\
y &= (QK^T \odot A^M \odot M) \cdot V_{dt} && \in \mathbb{R}^{H,N,d_h} \\
y_D &= y + D_{res} && \in \mathbb{R}^{H,N,d_h} \\
y'_D &= \text{vec}(y_D)_{(H,N,d_h) \longrightarrow (N,H \cdot d_h)} && \in \mathbb{R}^{N,H \cdot d_h} \\
y_N &= \text{RMSNorm}(y'_D \odot \sigma(Z)) && \in \mathbb{R}^{N,H \cdot d_h} \\
out &= y_N \cdot W_{out} && \in \mathbb{R}^{N,d}
\end{aligned}
$$

2.4 Softmax as a Recurrent Neural Network

Before explaining the changes to Mamba-2, it is necessary to motivate our rationale through prior work. (Mongaras and Larson [2025]) examine the accuracy discrepancy between linear attention and softmax attention and found that taking the sum of higher powers of higher order RNNs approaches the accuracy of full softmax attention. By the Taylor expansion of the exponential[4], softmax attention is equivalent to the sum of all nonnegative integer powers of the query-key inner product with the denominator being just a form of normalization. Taking the sum term-by-term, higher order terms produce a larger hidden state and improve downstream accuracy, approaching softmax accuracy.

While increasing the order of linear attention increases the expressiveness of the hidden state, it also increases the hidden state on the order of d_h^p where p is the order of the RNN and d_h is the head dimension. Vanilla linear attention turns out to be a first order approximation of softmax attention. That is, linear attention uses an additive RNN of order $p = 1$ which results in a hidden state of size (d_h, d_h). A higher order RNN would have a hidden state of roughly dimension (d_h^p, d_h), which can be reduced as explained in section 5.1.

Purely from a memory perspective, the hidden state of softmax attention is the KV cache, which requires $2 \times N \times d_h$ elements per head. The KV cache grows linearly with respect to the sequence length. While most values of p would result in an unreasonable hidden state size, a second order RNN, with $p = 2$ would result in a hidden state of dimension (d^2, d), still reasonable in memory for a sufficiently long context window. We use this intuition to improve Mamba-2 with a second order hidden state, improving the accuracy while keeping the memory requirements lower than softmax

[4] We note that the Taylor expansion used is more appropriately called the Maclaurin expansion, as the expansion point is centered on $x = 0$. However we adopt the same naming convention.

attention for a sufficiently large context window.

Additionally, normalization on the query-key sum requires a strictly non-negative image of the query-key inner product. A second order hidden state has an image with strictly non-negative values. Linear attention obtains positive values by applying a nonlinearity on the pre-image of the inner product of the queries and keys, thus restricting the domain of the inner product function. Squaring the query-key inner product presents no such restriction on the pre-image, while keeping the inner product image positive. As such, normalization on the queries and keys rather than on the attention output can be used, which has been shown to be both efficient and stable when using a form of online softmax attention (Milakov and Gimelshein [2018]) such as flash attention (Dao et al. [2022]).

3 Isolating Mamba-2 Accuracy Gains

To investigate the accuracy gains for each element in Mamba-2, we perform several ablations. For all our ablations, we start with the base Llama 2 (Touvron et al. [2023]) model and replace the softmax attention block with the Mamba-2 block. Our small model is about 300 million parameters. We use this model for all ablations, which helps to facilitate rapid experimentation. The medium model is about 700 million parameters and is used after ablation experiments to examine scalability of the algorithm. The maximum sequence length of all ablations is 2048 unless specified. For comparisons and reporting, we plot the test loss over $90K$ steps [5] and the final test loss at the end of training. Each model is trained to perform next token language modeling using the HuggingFace FineWeb dataset (CC-MAIN-2024-51 version) (Penedo et al. [2024]). This dataset is composed of over 15 Trillion clean and deduplicated tokens from CommonCrawl, a dataset of a crawl of webpages on the internet. As such, it is composed of various different types of data, from code, to random documents. Mamba-2 (Dao and Gu [2024]) was shown to be an expressive, powerful

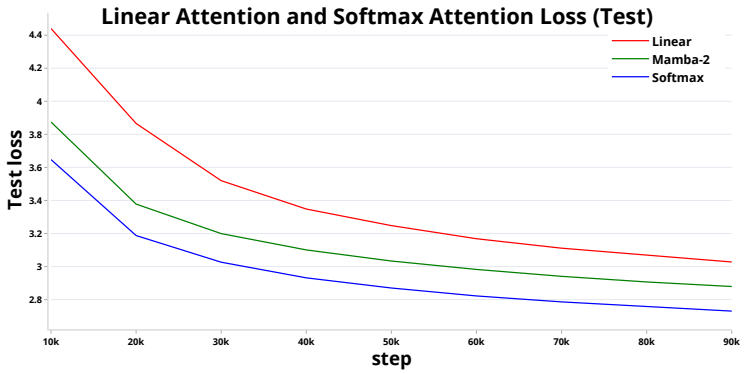


Figure 1: Accuracy of linear attention, Mamba, and softmax attention, keeping everything but the attention mechanism constant across experiments.

architecture. One that is both fast and accurate. Figure 1 shows the test loss for Mamba-2, normal linear attention, and softmax attention. As seen in Figure 1, Mamba-2 is significantly better than normal linear attention and is much closer in accuracy to softmax attention. However, most elements in the codebase were not ablated, leaving to question which components are important to the expressive power of Mamba-2 and which are unnecessary. We examined the official Mamba-2 repository [6] and isolate the components of the Mamba-2 block. Specifically, we ablate the following [7]:

1. QK activation type (SiLU, ReLU or None)
2. A-mask type (Original or Softplus)
3. Input convolution window size (window size one (no convolution), two, three, or four)
4. Additive D residual (binary, present or not)
5. Multiplicative Z gate (binary, present or not)
6. Normalization type (Output or Softmax)

[5] Our code trains for $100K$ steps and evaluates on the test loss every $10K$ steps. However, a bug exists which does the evaluation on 1 step after each $10K$ steps, not evaluating the $100K$'th step. We do not re-evaluate all experiments as it would take months of retraining and we believe $90K \rightarrow 100K$ would not lead to any insights.

[6] https://github.com/state-spaces/mamba

[7] Additional ablations found in our codebase. These are the most notable and interesting.

7. Value discretization dt (binary, present or not)

While most of the ablations we perform are straightforward, we want to highlight the normalization type and A-mask type. We call normalization on the QK inner product *softmax normalization*. Linear attention with a strictly positive inner product space can use softmax normalization, but Mamba-2 cannot as the queries and keys are not necessarily positive. Instead, Mamba-2 uses an RMS norm layer before the output projection. We call this form of normalization *output normalization*. The A-mask in the original Mamba-2 paper is tied to the values by the value discretization parameter, which we call *original A-masking*. We ablate if associating the A-mask with the discretization parameter is necessary. Because the A-mask must be strictly negative, as shown in equation (4), we test the negative softplus function on the A-mask, remove the discretization parameter and name this *softplus A-masking*.

$$\begin{aligned} &\text{“original” } A\text{-masking} \rightarrow A = -\exp(A_{log}) \odot dt \\ &\text{“softplus” } A\text{-masking} \rightarrow A = -\text{softplus}(A) \end{aligned} \tag{4}$$

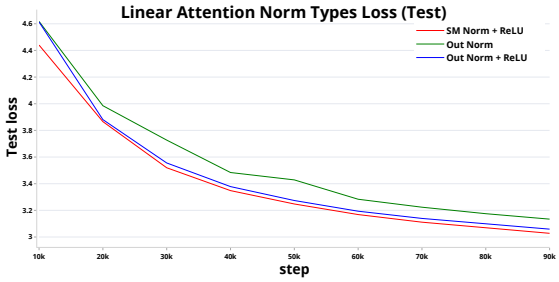


Figure 2: Accuracy of various norm types. Softmax normalization requires a positive inner-product space image, as such we use ReLU.

Model	Train loss	Test loss
ReLU	3.12	3.06
Conv (w=2)	2.87	2.93
Conv (w=3)	2.94	2.91
Conv (w=4)	**2.84**	2.9
Conv (w=2) + SiLU	2.95	2.9
Conv (w=3) + SiLU	2.95	2.89
Conv (w=4) + SiLU	2.9	**2.89**

Table 1: Adding a convolution to normal linear attention increases accuracy.

To begin our ablation study, we need a form of linear attention that will allow for more diverse experimentation. The original linear attention formulation as proposed by (Katharopoulos et al. [2020]) has softmax-style normalization on the QK inner product. This normalization restricts experimentation to a strictly non-negative inner product function range. Stability aside, as seen in Figure 2, adopting an output norm is nearly as accurate as softmax norm for basic linear attention. When ablating components of Mamba-2, we therefore use an output norm as it does not necessitate a strictly positive inner product space pre-image, giving more freedom to perform similar ablations. [8]

The first element of Mamba-2 we examine is the input convolution. As seen in Table 1, adding a convolution with kernel size 2 is significantly better than normal linear attention. Increasing the convolution size to 3 or adding an activation function (SiLU used in Mamba-2) gives a slight accuracy gain, but is not as impactful as adding the convolution itself. We only test up to a convolution size of 4 as the implementation of causal_conv_1d[9] only allows for a max window size of 4. As the window sizes increases beyond a window size of 2, the accuracy gains are small. To keep the implementation minimal, we adopt a convolution of window size 2 without an activation function. A window size of 2 minimizes the memory necessary in the complete attention operation. Increasing the convolution window requires storing $3d$ more memory per window size increase. That is, storing an additional past token for each of the queries, keys, and values. As for the activation function, the training loss appears to increase while the test loss decreases when adding the activation function. We opt to remove the activation function for simplicity, however adding it appears to slightly improve model test loss while increasing train loss.

[8] Since linear attention with QK normalization requires a strictly positive image of the QK inner product, some sort of activation function must be used. As such, we cannot test *SM Norm* by itself.

[9] https://github.com/Dao-AILab/causal-conv1d

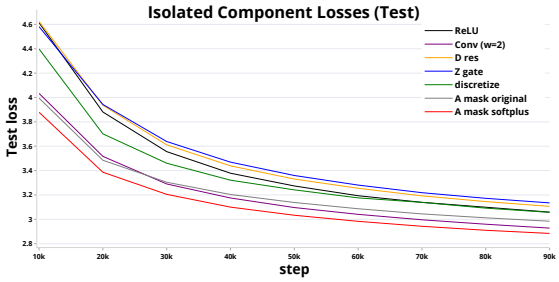
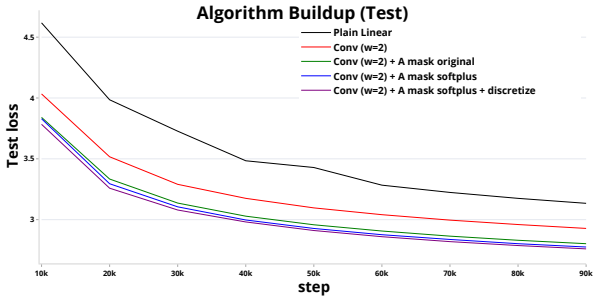


Figure 3: Isolated Mamba ablation

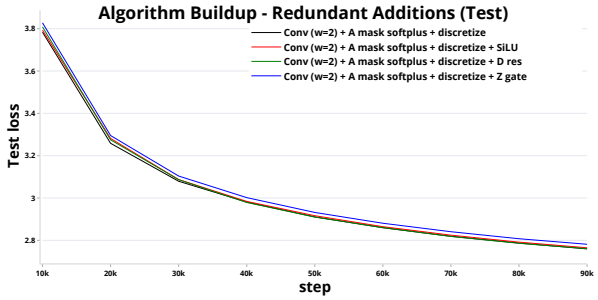
Model	Train loss	Test loss
SM Norm + ReLU	3.0	3.03
Out Norm	3.19	3.13
Out Norm + ReLU	3.12	3.06
Out Norm + Conv (w=2)	2.87	2.93
Out Norm + Conv (w=3)	2.94	2.91
Out Norm + Conv (w=2) + SiLU	2.95	2.9
Out Norm + D res	3.18	3.11
Out Norm + Z gate	3.2	3.13
Out Norm + value discretize	3.06	3.06
Out Norm + A-mask original	2.94	2.98
Out Norm + A-mask softplus	**2.85**	**2.88**

Table 2: Table of isolated component additions added to linear attention.

We ablate the rest of the components of Mamba-2 mentioned above in isolation. All isolated ablation test values can



(a) Main buildup



(b) Redundant components after buildup

Figure 4: Investigations of major and minor build ups in constructing the simplified Mamba-2 architecture.

be found in Table 2. Figure 3 also shows the results for ablating each component in isolation, graphically. The most impactful component was the A-mask, specifically the softplus variant. The original A-mask variant used in Mamba-2 is still quite impactful on model accuracy, but not as much as the softplus variant. The second most impactful component is the convolution. The rest of the isolated components result in minor accuracy gains compared to vanilla linear attention. We note that, of the collection of minor improvements, the value discretization method is most impactful. Even so, its impact is eclipsed by the A-mask and convolution components.

4 Building Up to the Mamba-2S Base Model

With each isolated component ablated, we want to buildup a minimal version of Mamba-2 that is as accurate as the original Mamba-2 algorithm (see Table 3). We start with the *output norm* variant of linear attention and build up the model on top of this base. While investigating all component interactions would be desirable, the number of combinations is too numerous. Instead, we build up the model using insights from the isolated component ablations. Figure 4a shows the main model buildup. To build up the new model, we start by combining the convolution and softplus A-mask as these two isolated components had the largest impact on accuracy, as shown in Figure 3. We find the combination of these two components results in higher accuracy than the isolated components themselves and use this as the new base model. We then investigate adding a third component, finding that adding the time discretization parameter gives a small boost in accuracy. In Figure 4b, we show that adding other components do not help increase accuracy of the model. In fact, adding a Z gate slightly reduces accuracy of the model. All buildup investigations can be found in Table 3.

To keep the model both simple and accurate, our resulting method uses the softplus A-mask, an input convolution of size 2, and time discretization. This simplified Mamba-2 algorithm, which we call Mamba-2S, only retains necessary components as shown in Algorithm 4.

Model	Train loss	Test loss
Plain Linear	3.19	3.13
Conv (w=2)	2.87	2.93
Conv (w=2) + A-mask original	2.81	2.8
Conv (w=2) + A-mask softplus	**2.67**	2.77
Conv (w=2) + A-mask softplus + value discretize	2.7	**2.76**
Conv (w=2) + A-mask softplus + value discretize + SiLU	2.73	2.76
Conv (w=2) + A-mask softplus + value discretize + D res	2.79	2.76
Conv (w=2) + A-mask softplus + value discretize + Z gate	2.72	2.78

Table 3: Table of all additions (buildups) leading up to our final algorithm. Italics convey the components in Mamba-2S.

Algorithm 4 Proposed Mamba-2 Simplified Forward (Mamba-2S): no setup besides weights

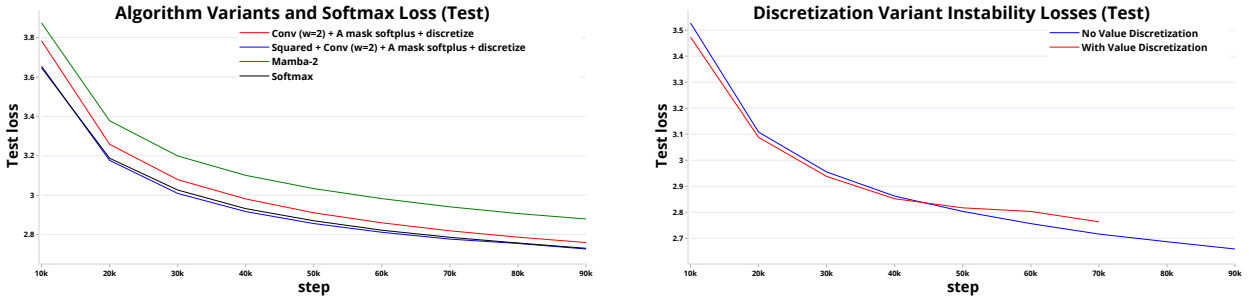
Require: $h \in \mathbb{R}^{N,d}, W_{QKV} \in \mathbb{R}^{d,3\cdot(H\cdot d_h)}, W_{dt} \in \mathbb{R}^{d,H}, W_A \in \mathbb{R}^{d,H}, W_{out} \in \mathbb{R}^{H\cdot d_h,d}$

$$
\begin{aligned}
Q, K, V &= \text{conv_1d}(h \cdot W_{QKV}) && \in \mathbb{R}^{H,N,d_h} \\
dt &= \text{softplus}(h \cdot W_{dt}) && \in \mathbb{R}^{H,N} \\
A &= -\text{softplus}(h \cdot W_A) && \in \mathbb{R}^{H,N} \\
V_{dt} &= V \odot dt && \in \mathbb{R}^{H,N,d_h} \\
A^{CS} &= \text{cumsum}(A) && \in \mathbb{R}^{H,N} \\
A^M &= \exp(A^{CS} - (A^{CS})^T) \quad \text{where } A^M_{ij} = \exp(A^{CS}_i - A^{CS}_j) && \in \mathbb{R}^{H,N,N} \\
M_{ij} &= \begin{cases} 1, & \text{if } i \geq j \\ 0, & \text{if } i < j \end{cases} && M \in \mathbb{R}^{H,N,N} \\
y &= (QK^T \odot A^M \odot M) \cdot V_{dt} && \in \mathbb{R}^{H,N,d_h} \\
y' &= \text{vec}(y)_{(H,N,d_h) \longrightarrow (N,H\cdot d_h)} && \in \mathbb{R}^{N,H\cdot d_h} \\
y_N &= \text{RMSNorm}(y') && \in \mathbb{R}^{N,H\cdot d_h} \\
out &= y_N \cdot W_{out} && \in \mathbb{R}^{N,d}
\end{aligned}
$$

5 Mamba-2 with a Squared Hidden State

While Mamba-2S is quite strong in accuracy as seen in Figure 5a, it still falls short of softmax attention. (Mongaras and Larson [2025]) showed that squaring the QK inner product results in a significant accuracy boost. Additionally, squaring the QK inner product results in a positive inner product space image. This space can adopt an online normalization algorithm like *softmax normalization*, which is more stable than *output normalization*. Using the simplified Algorithm 4, we find that squaring the QK inner product does result in a significant accuracy gain. As seen in Figure 5a, squaring the inner product results in softmax level accuracy while being more memory efficient than softmax attention for sufficient sequence length. Specifically, from Mamba-2S (Algo 4), we remove the RMSNorm, add softmax normalization, and replace the QK inner product with a squared inner product. We opt to use softmax normalization over output normalization as softmax normalization is more stable when using the online softmax algorithm (Milakov and Gimelshein [2018]). Online softmax normalization, like Flash Attention (Dao et al. [2022]), requires an exponential to update the max statistic over blocks. As the A-mask has an exponential, online softmax-like normalization can still be effective when the max statistic is stored in the A-mask as opposed to the e^{QK} inner product, as used in online softmax. The gradients for the forward and backward pass are supplied in Appendix C. Full Triton kernels can be found in our codebase, which are necessary for training.

The time discretization operation does improve the model, however the accuracy gains are minimal compared to adding the convolution and softplus A-mask. For smaller models, training is stable. For the medium model, we found that the magnitude of the values significantly increased across update steps, leading to numerical instability and divergence in training as seen in Figure 5b. Training a larger model with discretization can be somewhat stabilized—however this requires forcing triton to use IEEE FP32 input precision which is about 8x slower (NVIDIA Corporation [2020]) than the TF32 precision (a reduced precision mantissa compared to the standard IEEE FP32). A reasonable middle ground is a mix of input precision of FP32x3 for dot operations that need more precision and TF32 for others, which



(a) Small model performance comparison.

(b) Instability in the discretization variant.

Figure 5: Experimental results comparing accuracy and training stability.

is still slower than full TF32 precision but is slightly more stable. Training in pure FP32 PyTorch also results in stable training, but is considerably slower. While manipulating the kernel can result in more stable training, to keep the squared variant numerically stable for low precision kernels, we remove the time discretization operation for the medium model.

Our proposed squared algorithm, which we name 2Mamba, can be found in Algorithm 5. As seen in Figure 6, across various model sizes and sequence lengths, 2Mamba is nearly as accurate as softmax attention. One aspect worth further discussion is the placement of the square operation. We specifically place it on the QK inner product to increase the inner product dimension. An alternative placement could also include the A-mask. However, we find that including the A-mask does not result in any accuracy gains. While squaring after the A-mask application does result in a squared inner product on Q and K, it also results in squared terms in the A-mask, which is unnecessary. Intuitively, the A-mask should be applied directly to attention scores, which come from the squared inner product of Q and K.

Algorithm 5 2Mamba Algorithm

Require: $h \in \mathbb{R}^{N,d}, W_{QKV} \in \mathbb{R}^{d,3\cdot(H\cdot d_h)}, W_A \in \mathbb{R}^{d,H}, W_{out} \in \mathbb{R}^{H\cdot d_h,d}$

$$
\begin{aligned}
Q, K, V &= \text{conv_1d}(h \cdot W_{QKV}) && \in \mathbb{R}^{H,N,d_h} \\
A &= -\text{softplus}(h \cdot W_A) && \in \mathbb{R}^{H,N} \\
A^{CS} &= \text{cumsum}(A) && \in \mathbb{R}^{H,N} \\
A^M &= \exp(A^{CS} - (A^{CS})^T) \quad \text{where} \quad A^M_{ij} = \exp(A^{CS}_i - A^{CS}_j) && \in \mathbb{R}^{H,N,N}, \\
M_{ij} &= \begin{cases} 1, & \text{if } i \geq j \\ 0, & \text{if } i < j \end{cases} && M \in \mathbb{R}^{H,N,N} \\
y &= ((QK^T)^2 \odot A^M \odot M) \cdot V && \in \mathbb{R}^{H,N,d_h} \\
N &= \sum_j ((QK^T)^2 \odot A^M \odot M) && \in \mathbb{R}^{H,N} \\
y_N &= y/N && \in \mathbb{R}^{H,N,d_h} \\
y'_N &= \text{vec}(y_N)_{(H,N,d_h) \longrightarrow (N,H\cdot d_h)} && \in \mathbb{R}^{N,H\cdot d_h} \\
out &= y'_N \cdot W_{out} && \in \mathbb{R}^{N,d}
\end{aligned}
$$

5.1 2Mamba Algorithm Efficiency

While squaring the inner product does result in a large accuracy gain, we note that the recurrent state requires approximately the square of the space as the original linear variant. When squaring the inner product or taking the Kronecker product of Q and K, and then performing the inner product, some terms are repeated due to the commutativity of the multiplication operation in the inner product. This means that as opposed to doing a Kronecker product on the queries and keys, one can do an operation that calculates all unique second-order product terms and calculate their multiplicity via the multinomial theorem. We provide a Triton kernel for creating a vector of second order products of a given

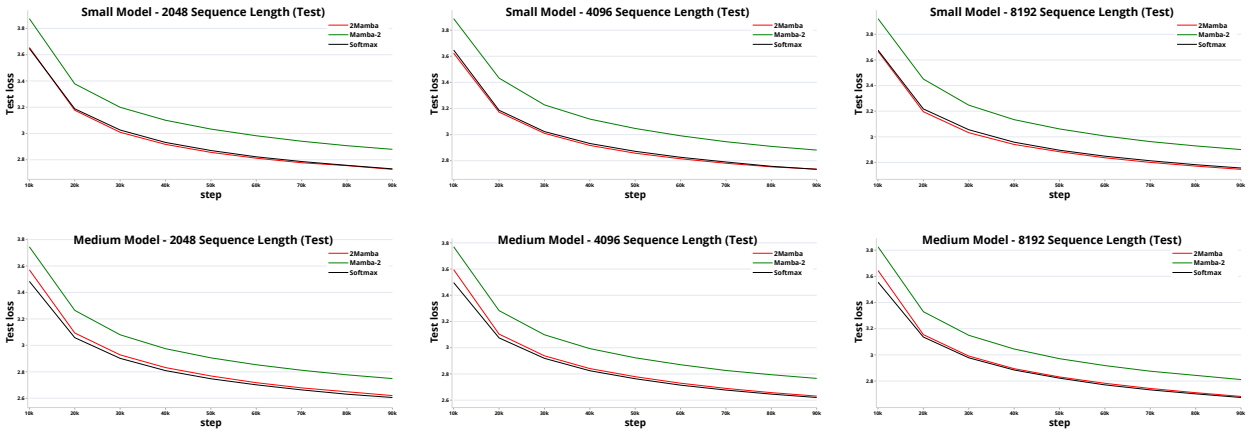


Figure 6: **First row**: Small model (∼300M params) test loss for Mamba-2, 2Mamba, and softmax attention trained on 2048, 4096, and 8192 sequence lengths. **Second row**: Medium model (∼700M params) test loss for Mamba-2, our proposed algorithm, and softmax attention trained on 2048, 4096, and 8192 sequence lengths

vector.[10] The number of terms, as seen in equation (5) is equal to $\frac{d(d+1)}{2}$, which is less than half the elements within the vector obtained from naively calculating all d^2 elements.[11]

$$num_terms = \binom{2+d-1}{d-1} = \binom{d+1}{d-1} = \frac{(d+1)!}{2(d-1)!} = \frac{(d+1)\cdot d\cdot (d-1)!}{2(d-1)!} = \frac{d\cdot(d+1)}{2} \tag{5}$$

As the complexity of softmax attention is linear in memory due to the KV cache, there exists a sequence length such that squaring the inner product requires less memory than softmax attention. A KV cache requires memory on the order of $2Ld$ for a single attention head. By using equation (5) for the key dimension and multiplying by d for the value dimension, a hidden state for squared linear attention requires memory on the order of $d^2(d+1)/2$ per attention head. Adding a convolution adds an extra $3d$ memory per attention head and using softmax normalization adds an extra $d(d+1)/2$ memory per attention head. In total, this results in a hidden state size of $\frac{d(d+1)^2}{2} + 3d$ per attention head. The inequality in equation (6) provides the necessary sequence length, N, such that any sequence length longer than N is more memory efficient using a second-order hidden state than using softmax attention.

$$2Nd > \frac{d(d+1)^2}{2} + 3d \qquad \longrightarrow \qquad N > \frac{(d+1)^2}{4} + \frac{3}{2} \tag{6}$$

In practice, we used a head dimension of 64. According to inequality in equation (6), a sequence length greater than $\frac{(64+1)^2}{4} + \frac{3}{2} \approx 1058$ would result in a lower memory usage than softmax attention. To ensure the squared model retains good accuracy beyond this sequence length, we examine models with a context size up to 8192 tokens. As such, squaring the QK inner product is less memory constrained than softmax attention past the sequence length of 1058 while matching softmax level accuracy. This memory result is verified by running both softmax attention and 2Mamba while accumulating the raw hidden state memory usage from each model, as seen in figure 7. Our codebase has a state caching inference implementation for both algorithms. The inference algorithm is also supplied in Appendix A.

5.2 Effective Context Usage

To confirm 2Mamba uses its context effectively, we employ the The Needle in a Haystack (NIAH) test, which measures ability of the model to retrieve a specific, relevant fact (the "needle") inserted into a long prompt (the "haystack"). That is, it assesses how well a model recalls information placed at different locations within large prompts. In our testing, we train a model for $400,000$ steps on a batch size of 64 with a maximum context size of $8,192$ tokens. During this long training run, we plot the test loss, as seen in Figure 9, showing that 2Mamba is still competitive with softmax attention for long training runs. After training for $400,000$ steps, we evaluate our model on Nanotron's needle in a haystack test[12] on context lengths from about $1,024$ characters to about $16,384$ characters. This test hides a password inside a large corpus of irrelevant text. The model must memorize the password, and repeat it after parsing the large

[10] https://github.com/gmongaras/Triton-Efficient-Kronecker-Product

[11] For notational brevity, we denote the head dimension as d in this section.

[12] https://huggingface.co/datasets/nanotron/simple_needle_in_a_hay_stack

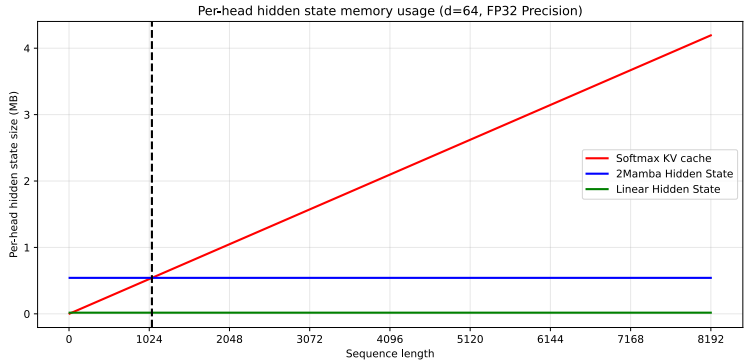


Figure 7: Memory usage of a single head of softmax attention, 2Mamba, and linear attention, utilizing softmax-like normalization and a convolution with kernel size 2.

corpus of text. This benchmark tests how well a model uses its context window and for how long it can store important information. As seen in Figure 8, 2Mamba method is slightly better than softmax in context retrieval, and much better than Mamba-2. This result provides evidence that 2Mamba extends beyond NTP tasks and can utilize its context effectively.

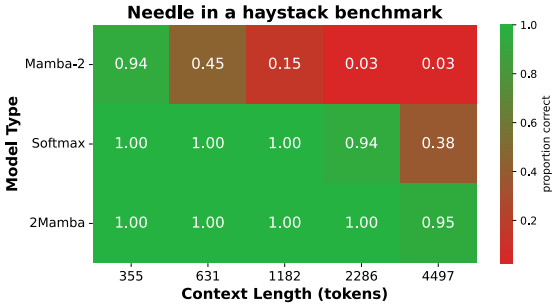


Figure 8: One-shot NIAH benchmark results using maximum likelihood sampling over 1000 sequences. Green indicates the predicted needle was correct more often while red indicates the predicted needle was predicted less often. Scores are indicative of the proportion of sequences in which the model predicted the needle correctly.

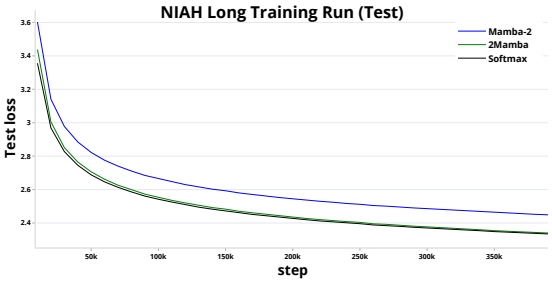


Figure 9: Long training run of various models with a batch size of 64 on $8,192$ max sequence length.

6 2Mamba With an Exponentiated Hidden State

The property of squaring the inner product was used to make Mamba-2 more accurate while remaining efficient. As shown in (Mongaras and Larson [2025]), adding higher orders typically results in additional accuracy gains (though diminishing with each new term). In the limiting case, one can use the exponential function which results in softmax attention. Therefore, we ask: If the exponential function on normal linear attention gives softmax attention, then what would happen if the QK inner product of 2Mamba was exponentiated as opposed to squared? As seen in Figure 10, exponentiating the inner product of Q and K results in slightly better accuracy than softmax attention. We name the exponentiated form 2Mamba-E. One can interpret this algorithm as either exponentiating 2Mamba or as softmax attention with the softplus A-mask and input convolution. The latter explanation seems more intuitive and satisfying when considering traditional transformer literature. Adding an A-mask, or decay mask, to softmax attention has also been proposed in the Forgetting Transformer (Lin et al. [2025]). The main difference between the forgetting transformer and the algorithm we use is the softplus A-mask construction. The forgetting transformer uses $\log(\text{sigmoid}(x))$ as the function for the softplus A-mask values while we use $-\text{softplus}(-x)$. It turns out these are

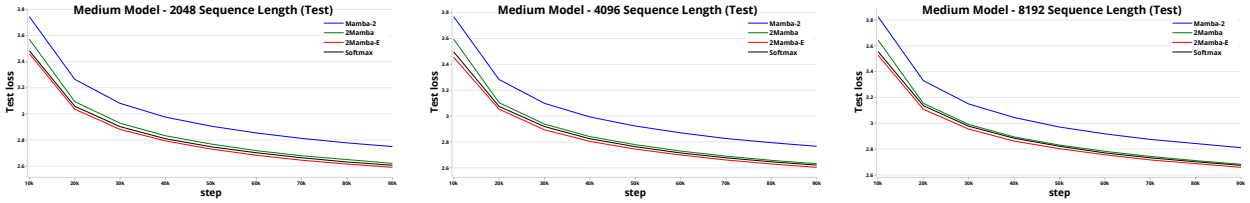


Figure 10: Medium model (∼700M params) test loss for Mamba-2, 2Mamba-E, and softmax attention trained on 2048, 4096, and 8192 sequence lengths.

nearly equivalent (only differing by sign) as seen in equation 7.

$$\log(\text{sigmoid}(x)) = \log\left(\frac{1}{1+e^{-x}}\right) = -\log(1+e^{-x}) = -\text{softplus}(-x) \tag{7}$$

From this perspective, our work on exponentiated hidden states also provides an intuitive connection between SSM models, like Mamba-2, and transformer variants like the forgetting transformer.

7 Conclusion and Future Work

This work examines the importance of each individual component of Mamba-2 to build up a minimal implementation, Mamba-2S. From this base model, we take the second order hidden state to achieve a model, 2Mamba, that is as accurate as softmax attention, yet constant in memory. Additionally, we show that our implementation, when exponentiated, results in an architecture, 2Mamba-E, that is better than softmax attention and show that this implementation is similar to the forgetting transformer (Lin et al. [2025]). 2Mamba is a step forward in bridging the gap between linear attention architectures and softmax attention, while being more efficient in practice.

The exploration of this work closely examined the architecture choices made in Mamba-2 (Dao and Gu [2024]). Another parallel work that improves linear attention is DeltaNet (Yang et al. [2024b]). Mamba-2 and DeltaNet have been combined in Gated DeltaNet (Yang et al. [2025]). Similarly, future work can look into adding DeltaNet to 2Mamba to further improve the algorithm.

Future work can also look into the hidden state created by squaring the inner product of the queries and keys. While we stick with a static hidden state size, future work can examine how varying the hidden state size affects downstream accuracy and if squaring the inner product space results in better or similar accuracy than using a query and key projection to the same size. This future work can make 2Mamba more efficient in terms of memory by optimizing the hidden state size.

References

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints, 2023. URL `https://arxiv.org/abs/2305.13245`.

Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate, 2016. URL `https://arxiv.org/abs/1409.0473`.

Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Tomas Jackson, Sally Jesmonth, Nikhil J Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Kuang-Huei Lee, Sergey Levine, Yao Lu, Utsav Malla, Deeksha Manjunath, Igor Mordatch, Ofir Nachum, Carolina Parada, Jodilyn Peralta, Emily Perez, Karl Pertsch, Jornell Quiambao, Kanishka Rao, Michael Ryoo, Grecia Salazar, Pannag Sanketi, Kevin Sayed, Jaspiar Singh, Sumedh Sontakke, Austin Stone, Clayton Tan, Huong Tran, Vincent Vanhoucke, Steve Vega, Quan Vuong, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich. Rt-1: Robotics transformer for real-world control at scale, 2023. URL `https://arxiv.org/abs/2212.06817`.

Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. In *ICML 2021 Workshop on Unsupervised Reinforcement Learning*, 2021. URL `https://openreview.net/forum?id=gaCGNwsWITG`.

Krzysztof Marcin Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Quincy Davis, Afroz Mohiuddin, Lukasz Kaiser, David Benjamin Belanger, Lucy J Colwell, and Adrian Weller. Rethinking attention with performers. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=Ua6zuk0WRH.

Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 10041–10071. PMLR, 21–27 Jul 2024. URL https://proceedings.mlr.press/v235/dao24a.html.

Tri Dao, Daniel Y Fu, Stefano Ermon, Atri Rudra, and Christopher Re. Flashattention: Fast and memory-efficient exact attention with IO-awareness. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho, editors, *Advances in Neural Information Processing Systems*, 2022. URL https://openreview.net/forum?id=H4DqfPSibmx.

Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=YicbFdNTTy.

Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The pile: An 800gb dataset of diverse text for language modeling. *CoRR*, abs/2101.00027, 2021. URL https://arxiv.org/abs/2101.00027.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *First Conference on Language Modeling*, 2024. URL https://openreview.net/forum?id=tEYskw1VY2.

Albert Gu, Karan Goel, and Christopher Re. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=uYLFoz1vlAC.

John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, Alex Bridgland, Clemens Meyer, Simon A. A. Kohl, Andrew J. Ballard, Andrew Cowie, Bernardino Romera-Paredes, Stanislav Nikolov, Rishub Jain, Jonas Adler, Trevor Back, Stig Petersen, David Reiman, Ellen Clancy, Michal Zielinski, Martin Steinegger, Michalina Pacholska, Tamas Berghammer, Sebastian Bodenstein, David Silver, Oriol Vinyals, Andrew W. Senior, Koray Kavukcuoglu, Pushmeet Kohli, and Demis Hassabis. Highly accurate protein structure prediction with alphafold. *Nature*, 596(7873):583–589, Aug 2021. ISSN 1476-4687. doi: 10.1038/s41586-021-03819-2. URL https://doi.org/10.1038/s41586-021-03819-2.

R. E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 03 1960. ISSN 0021-9223. doi: 10.1115/1.3662552. URL https://doi.org/10.1115/1.3662552.

Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5156–5165. PMLR, 13–18 Jul 2020. URL https://proceedings.mlr.press/v119/katharopoulos20a.html.

Weijie Kong, Qi Tian, Zijian Zhang, Rox Min, Zuozhuo Dai, Jin Zhou, Jiangfeng Xiong, Xin Li, Bo Wu, Jianwei Zhang, Kathrina Wu, Qin Lin, Junkun Yuan, Yanxin Long, Aladdin Wang, Andong Wang, Changlin Li, Duojun Huang, Fang Yang, Hao Tan, Hongmei Wang, Jacob Song, Jiawang Bai, Jianbing Wu, Jinbao Xue, Joey Wang, Kai Wang, Mengyang Liu, Pengyu Li, Shuai Li, Weiyan Wang, Wenqing Yu, Xinchi Deng, Yang Li, Yi Chen, Yutao Cui, Yuanbo Peng, Zhentao Yu, Zhiyu He, Zhiyong Xu, Zixiang Zhou, Zunnan Xu, Yangyu Tao, Qinglin Lu, Songtao Liu, Dax Zhou, Hongfa Wang, Yong Yang, Di Wang, Yuhong Liu, Jie Jiang, and Caesar Zhong. Hunyuanvideo: A systematic framework for large video generative models, 2025. URL https://arxiv.org/abs/2412.03603.

Zhixuan Lin, Evgenii Nikishin, Xu He, and Aaron Courville. Forgetting transformer: Softmax attention with a forget gate. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=q2Lnyegkr8.

Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. *CoRR*, abs/1805.02867, 2018. URL http://arxiv.org/abs/1805.02867.

Gabriel Mongaras and Eric C. Larson. On the expressiveness of softmax attention: A recurrent neural network perspective. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL https://openreview.net/forum?id=PHcITOi3vV.

NVIDIA Corporation. Nvidia a100 tensor core gpu architecture. Technical report, NVIDIA Corporation, May 2020. URL https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf. v1.0.

Guilherme Penedo, Hynek Kydlíček, Loubna Ben allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The fineweb datasets: Decanting the web for the finest text data at scale. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024. URL https://openreview.net/forum?id=n6SCkn2QaG.

Zhen Qin, Weixuan Sun, Hui Deng, Dongxu Li, Yunshen Wei, Baohong Lv, Junjie Yan, Lingpeng Kong, and Yiran Zhong. cosformer: Rethinking softmax in attention. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=Bl8CQrx2Up4.

Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine Mcleavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 28492–28518. PMLR, 23–29 Jul 2023. URL https://proceedings.mlr.press/v202/radford23a.html.

Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. Hierarchical text-conditional image generation with clip latents, 2022. URL https://arxiv.org/abs/2204.06125.

David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. doi: 10.1038/323533a0. URL https://doi.org/10.1038/323533a0.

Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 9355–9366. PMLR, 18–24 Jul 2021. URL https://proceedings.mlr.press/v139/schlag21a.html.

Zhiqiang Shen, Tianhua Tao, Liqun Ma, Willie Neiswanger, Zhengzhong Liu, Hongyi Wang, Bowen Tan, Joel Hestness, Natalia Vassilieva, Daria Soboleva, and Eric Xing. SlimPajama-DC: Understanding Data Combinations for LLM Training. *arXiv e-prints*, art. arXiv:2309.10818, September 2023. doi: 10.48550/arXiv.2309.10818.

Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models, 2023. URL https://arxiv.org/abs/2307.08621.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023. URL https://arxiv.org/abs/2307.09288.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 56501–56523. PMLR, 21–27 Jul 2024a. URL https://proceedings.mlr.press/v235/yang24ab.html.

Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024b. URL https://openreview.net/forum?id=y8Rm4VNRPH.

Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=r8H7xhYPwz.

Michael Zhang, Kush Bhatia, Hermann Kumbong, and Christopher Re. The hedgehog & the porcupine: Expressive linear attentions with softmax mimicry. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=4g02l2N2Nx.

A Inference Algorithm

Inference requires keeping around a few hidden states. $H^{\uparrow}_{t-1}$ is the numerator, keeping all key-value products. $H^{\downarrow}_{t-1}$ is the denominator, only keeping the keys and is only necessary for softmax-like normalization. With an input convolution, one must keep around $window_size - 1$ number of queries, keys, and values. The code below shows a window size of 2, so only one query, key, and value must be cached.

Algorithm 6 2Mamba Inference

Require: $h \in \mathbb{R}^d, W_{QKV} \in \mathbb{R}^{d,3\cdot(H\cdot d_h)} W_A \in \mathbb{R}^{d,H}, W_{out} \in \mathbb{R}^{H\cdot d_h,d}$
Require: $H^{\uparrow}_{t-1} \in \mathbb{R}^{H,D,d_h}, H^{\downarrow}_{t-1} \in \mathbb{R}^{H,D}, q_{t-1} \in \mathbb{R}^{H,d_h}, k_{t-1} \in \mathbb{R}^{H,d_h}, v_{t-1} \in \mathbb{R}^{H,d_h}$
Require: $D = \frac{d_h(d_h+1)}{2} \leq d_h^2$

$$
\begin{aligned}
q_t, k_t, v_t &= h_t W_{QKV} && \in \mathbb{R}^{H,d_h} \\
q^c_t, k^c_t, v^c_t &= \text{conv_1d}(q_t, k_t, v_t, q_{t-1}, k_{t-1}, v_{t-1}, \text{window_size} = 2) && \in \mathbb{R}^{H,d_h} \\
a_t &= exp(-softplus(h_t \cdot W_A)) && \in \mathbb{R}^{H} \\
(q^c_t)^{\otimes 2} &= q^c_t \otimes q^c_t && \in \mathbb{R}^{H,D} \\
(k^c_t)^{\otimes 2} &= k^c_t \otimes k^c_t && \in \mathbb{R}^{H,D} \\
H^{\uparrow}_t &= H^{\uparrow}_{t-1} \odot a_t + \left[(k^c_t)^{\otimes 2}\right]^T \cdot (v^c_t) && \in \mathbb{R}^{H,D,d_h} \\
H^{\downarrow}_t &= H^{\downarrow}_{t-1} \odot a_t + \left[(k^c_t)^{\otimes 2}\right]^T && \in \mathbb{R}^{H,D} \\
y_N &= \frac{(q^c_t)^{\otimes 2} \cdot H^{\uparrow}_t}{(q^c_t)^{\otimes 2} \cdot H^{\downarrow}_t} && \in \mathbb{R}^{H,d_h} \\
o_t &= y_t \cdot W_{out} && \in \mathbb{R}^{d}
\end{aligned}
$$

B Model Ablation Details

In this section, we briefly mention the setup for all ablations. Specifically, the llama 2 (Touvron et al. [2023]) architecture is used as our base model. To ablate the changes to the attention mechanism, we replace the attention mechanism in all layers of the llama 2 model with the ablated variations, keeping all other parts of the architecture constant. The base max sequence length is 2048, however 4096 and 8192 sequence lengths are used for testing models on longer context lengths. Most experiments are done with a base model of about 300 million parameters. To test scale we increase the size to 700 million parameters. The following is information regarding our training setup:

- dataset: HuggingFaceFW/fineweb (CC-MAIN-2024-51 version)
- test dataset: held out 0.1 percent of the entire dataset (seed 123)
- model name: meta-llama/Llama-2-7b-hf
- optimizer: AdamW (0.9, 0.999 betas)
- batch size (over all devices): 32
- learning rate: 1e-4
- warmup steps: 10,000

- total steps: 100,000
- weight decay: 0.01
- no grad clipping

The 300 million parameter model (small) has the following setup:

- hidden size: 1024
- intermediate MLP size: 2048
- hidden MLP activation: silu
- num attention heads: 16 (64 head dim)
- num layers: 20
- vocab size: 32,000 (llama 2 vocab)

The 700 million parameter model (medium) has the following setup:

- hidden size: 1536
- intermediate MLP size: 3072
- hidden MLP activation: silu
- num attention heads: 24 (64 head dim)
- num layers: 27
- vocab size: 32,000 (llama 2 vocab)

C Gradients

We derive the gradient equations for all necessary kernels and provide them in this section. Each gradient assumes the input is the query, key, and value matrices and the output is the post-attention outputs. Each kernel performs the attention operation, but no pre-processing or post-processing.

C.1 Linear

$$
\begin{aligned}
O &= \left(QK^T \odot M\right) V \\
\frac{\partial L}{\partial Q} &= \left(\frac{\partial L}{\partial O} V^T \odot M\right) K \\
\frac{\partial L}{\partial K} &= \left(V \frac{\partial L}{\partial O}^T \odot M^T\right) Q \\
\frac{\partial L}{\partial V} &= \left(QK^T \odot M\right)^T \frac{\partial L}{\partial O}
\end{aligned}
$$

C.2 Linear with Softmax Norm

$$
\begin{aligned}
O &= \frac{QK^T \odot M}{\sum (QK^T \odot M)} V \\
Y &= QK^T \odot M \\
S &= \sum_j Y \\
Y_N &= Norm(Y) = \frac{Y}{N} = \frac{Y}{\sum Y} \\
G &= \frac{\partial L}{\partial O} V^T \odot M \\
D &= \frac{G - \sum_j (Y_N \odot G)}{S} \odot M \\
\frac{\partial L}{\partial Q} &= DK \\
\frac{\partial L}{\partial K} &= D^T Q \\
\frac{\partial L}{\partial V} &= Y_N^T \frac{\partial L}{\partial O}
\end{aligned}
$$

C.3 Linear with A-Gate

Note: Typically we use M to denote the causal mask. Here, we additionally use M to denote the row (key and value) axis whereas the column (query) axis is denoted by N. Additionally, $\sum_j$ denotes a summation along the M axis while $\sum_i$ denotes a summation along the N axis. The differentiation is necessary for the gradients of the A-mask. In the self-attention case, $N = M$, however we still make this differentiation for the gradient derivation below.

$$
\begin{aligned}
A_M &= e^{A - A^T} \quad \in \mathbb{R}^{B,H,N,M}, \quad A \in \mathbb{R}^{B,H,N} \\
O &= \left(QK^T \odot M \odot A_M\right) V \\
\frac{\partial L}{\partial Q} &= \left(\frac{\partial L}{\partial O} V^T \odot M \odot A_M\right) K \\
\frac{\partial L}{\partial K} &= \left(\frac{\partial L}{\partial O} V^T \odot M \odot A_M\right)^T Q \\
\frac{\partial L}{\partial V} &= \left(QK^T \odot M \odot A_M\right)^T \frac{\partial L}{\partial O} \\
\frac{\partial L}{\partial A_M^N} &= \sum_j \left(QK^T \odot M \odot A_M \odot \frac{\partial L}{\partial O} V^T\right) \\
\frac{\partial L}{\partial A_M^M} &= -\sum_i \left(QK^T \odot M \odot A_M \odot \frac{\partial L}{\partial O} V^T\right) \\
\frac{\partial L}{\partial A_M} &= \frac{\partial L}{\partial A_M^N} + \frac{\partial L}{\partial A_M^M}
\end{aligned}
$$

C.4 Squared with A-mask

$$
\begin{aligned}
A_M &= e^{A-A^T} \quad \in \mathbb{R}^{B,H,N,M}, \quad A \in \mathbb{R}^{B,H,N} \\
O &= \left((QK^T)^2 \odot M \odot A_M\right) V \\
\frac{\partial L}{\partial Q} &= \left(2 \odot QK^T \odot \frac{\partial L}{\partial O} V^T \odot M \odot A_M\right) K \\
\frac{\partial L}{\partial K} &= \left(2 \odot QK^T \odot \frac{\partial L}{\partial O} V^T \odot M \odot A_M\right)^T Q \\
\frac{\partial L}{\partial V} &= \left((QK^T)^2 \odot M \odot A_M\right)^T \frac{\partial L}{\partial O} \\
\frac{\partial L}{\partial A_M^N} &= \sum_j \left((QK^T)^2 \odot M \odot A_M \odot \frac{\partial L}{\partial O} V^T\right) \\
\frac{\partial L}{\partial A_M^M} &= -\sum_i \left((QK^T)^2 \odot M \odot A_M \odot \frac{\partial L}{\partial O} V^T\right) \\
\frac{\partial L}{\partial A_M} &= \frac{\partial L}{\partial A_M^N} + \frac{\partial L}{\partial A_M^M}
\end{aligned}
$$

C.5 Squared with A-mask and Softmax Norm (2Mamba)

$$
\begin{aligned}
O &= \frac{\left(QK^T\right)^2 \odot M \odot A_M}{\sum \left(\left(QK^T\right)^2 \odot M \odot A_M\right)} V \\
Y &= \left(QK^T\right)^2 \odot M \odot A_M \\
S &= \sum_j Y \\
Y_N &= Norm(Y) = \frac{Y}{S} \\
G &= V \frac{\partial L}{\partial O}^T \odot A_M \odot M - \sum_j \left(Y_N \odot \frac{\partial L}{\partial O} V^T\right) \\
D &= \frac{2 \odot QK^T \odot G}{S} \odot M
\end{aligned}
$$

$$
\begin{aligned}
\frac{\partial L}{\partial Q} &= DK \\
\frac{\partial L}{\partial K} &= D^T Q \\
\frac{\partial L}{\partial V} &= Y_N^T \frac{\partial L}{\partial O} \\
\frac{\partial L}{\partial A_M^N} &= \sum_j (Y_N \odot G) \\
\frac{\partial L}{\partial A_M^M} &= -\sum_i (Y_N \odot G) \\
\frac{\partial L}{\partial A_M} &= \frac{\partial L}{\partial A_M^N} + \frac{\partial L}{\partial A_M^M}
\end{aligned}
$$

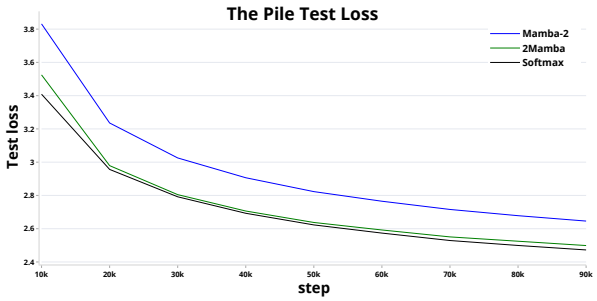
C.6 Exponentiated with A-mask and Softmax Norm (2Mamba-E)

$$
\begin{aligned}
O &= \frac{e^{QK^T} \odot M \odot A_M}{\sum \left(e^{QK^T} \odot M \odot A_M\right)} V \\
Y &= e^{QK^T} \odot M \odot A_M \\
S &= \sum_j Y \\
Y_N &= Norm(Y) = \frac{Y}{S} \\
G &= \frac{\partial L}{\partial O} V^T \odot M - \sum_{d_h} \left(O \odot \frac{\partial L}{\partial O}\right) \\
D &= Y_N \odot G \odot M
\end{aligned}
$$

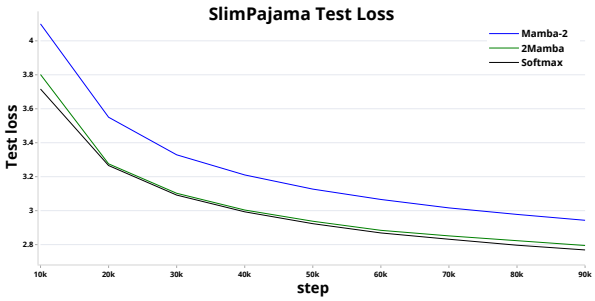
$$
\begin{aligned}
\frac{\partial L}{\partial Q} &= DK \\
\frac{\partial L}{\partial K} &= D^T Q \\
\frac{\partial L}{\partial V} &= Y_N^T \frac{\partial L}{\partial O} \\
\frac{\partial L}{\partial A_M^N} &= \sum_j D \\
\frac{\partial L}{\partial A_M^M} &= -\sum_i D \\
\frac{\partial L}{\partial A_M} &= \frac{\partial L}{\partial A_M^N} + \frac{\partial L}{\partial A_M^M}
\end{aligned}
$$

D Pile and SlimPJ Loss Curves

To verify 2Mamba generalizes to other datasets, we perform a training run with medium sized models on The Pile (Gao et al. [2021]) and SlimPajama (Shen et al. [2023]).



(a) Test loss on The Pile



(b) Test loss on SlimPajama